ICT 3101: Operating System

Chapter 10:

VIRTUAL MEMORY

Md Mahmudur Rahman
Assistant Professor, IIT

PROCESSES | MEMORY MANAGEMENT | SCHEDULING | FILE MANAGEMENT

Background

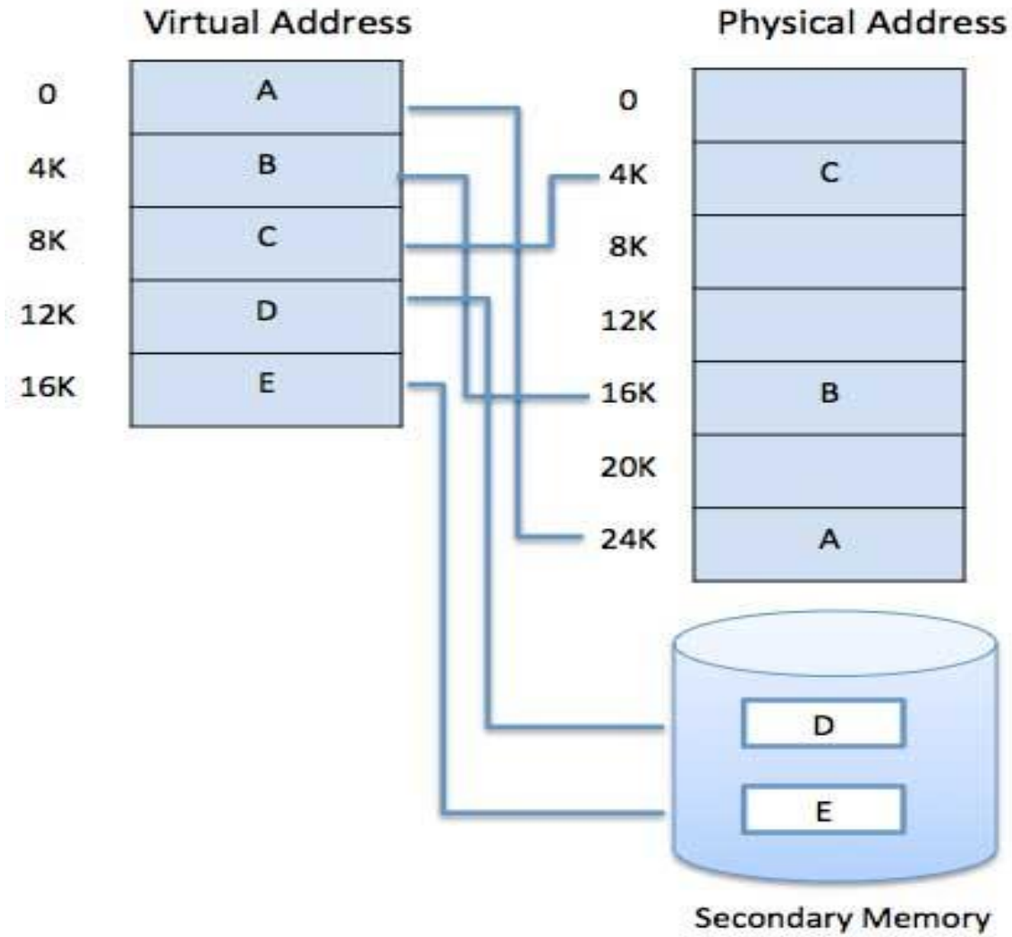
- Code needs to be in memory to execute, but entire program rarely used.
- Entire program code not needed at same time.
- Consider ability to execute partially-loaded program:
  - User written error handling routines are used only when an error occurred in the data or computation.
  - Certain options and features of a program may be used rarely.
  - Many tables are assigned a fixed amount of address space even though only a small amount of the table is actually used.
  - The ability to execute a program that is only partially in memory would counter many benefits.
  - Less number of I/O would be needed to load or swap each user program into memory.
  - A program would no longer be constrained by the amount of physical memory that is available.
  - Each user program could take less physical memory, more programs could be run the same time, with a corresponding increase in CPU utilization and throughput.

Virtual Memory

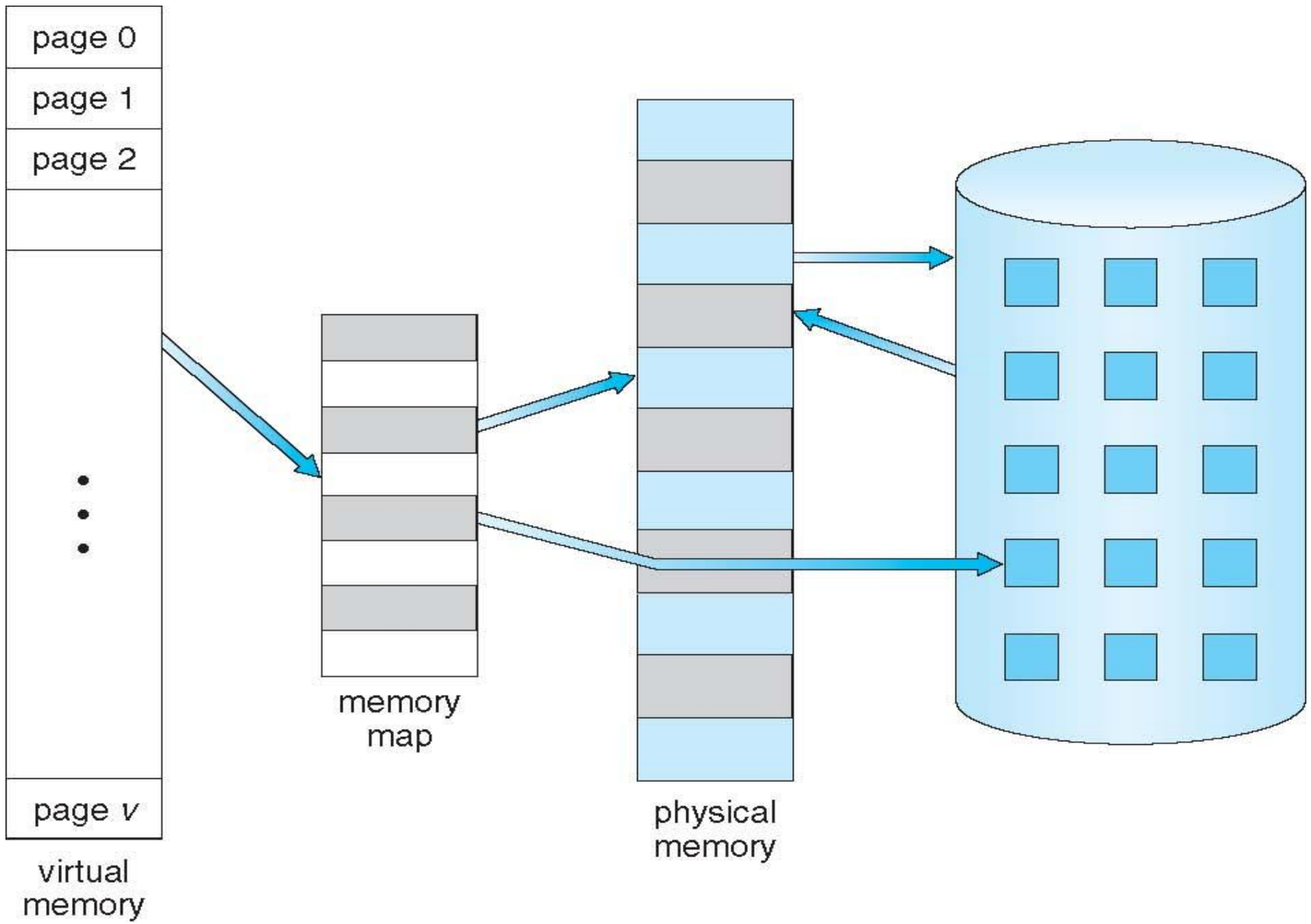
- Virtual Memory is a storage scheme that provides user an illusion of having a very big main memory. This is done by treating a part of secondary memory as the main memory.
- In this scheme, User can load the bigger size processes than the available main memory by having the illusion that the memory is available to load the process.
- Instead of loading one big process in the main memory, the Operating System loads the different parts of more than one process in the main memory.
- By doing this, the degree of multiprogramming will be increased and therefore, the CPU utilization will also be increased.
- **Virtual address space** – logical view of how process is stored in memory.
  - Usually start at address 0, contiguous addresses until end of space.
  - Meanwhile, physical memory organized in page frames.
  - MMU must map logical to physical address.

How Virtual Memory Works?

- In modern word, virtual memory has become quite common these days.
- In this scheme, whenever some pages needs to be loaded in the main memory for the execution and the memory is not available for those many pages, then in that case, instead of stopping the pages from entering in the main memory, the OS search for the RAM area that are least used in the recent times or that are not referenced and copy that into the secondary memory to make the space for the new pages in the main memory.
- Since all this procedure happens automatically, therefore it makes the computer feel like it is having the unlimited RAM.
- Virtual memory can be implemented via:
  - Demand paging
  - Demand segmentation

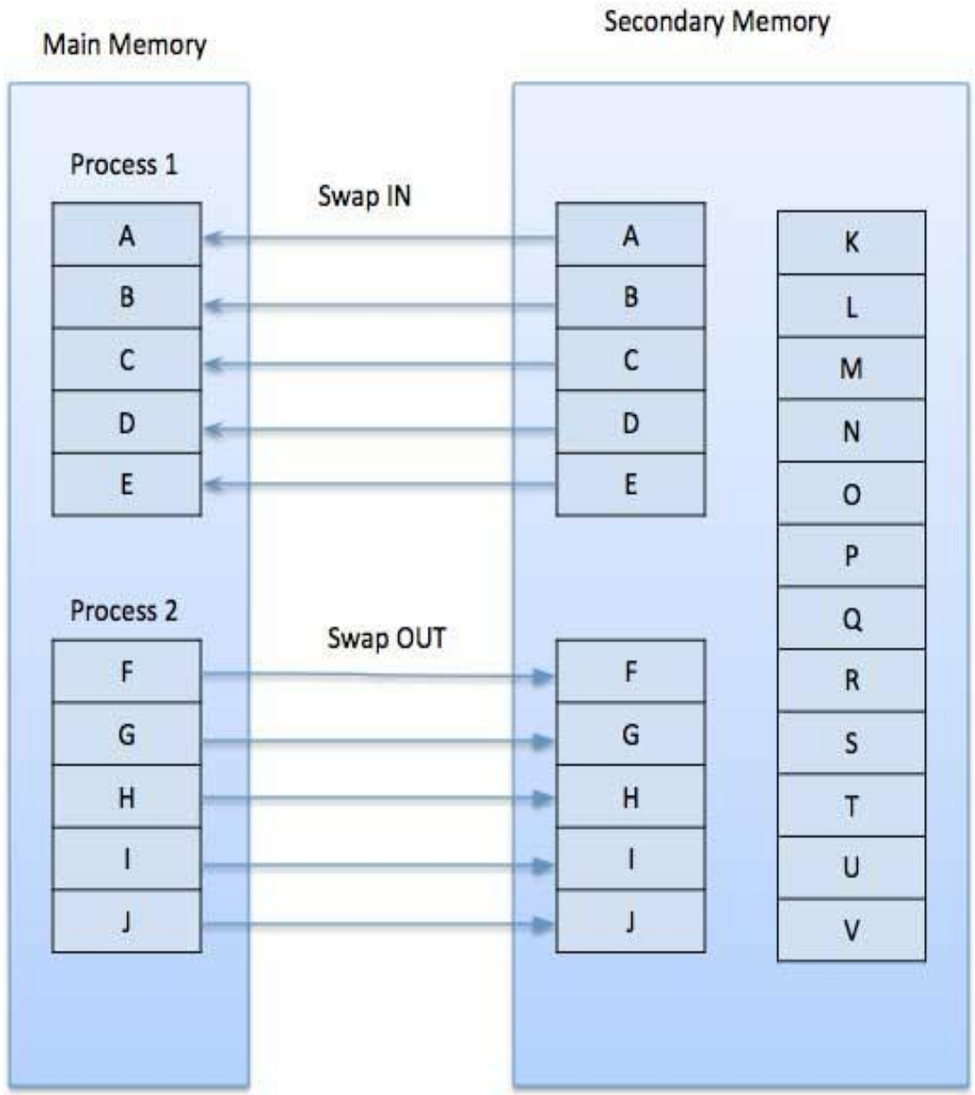


Virtual Memory is Larger Than Physical Memory



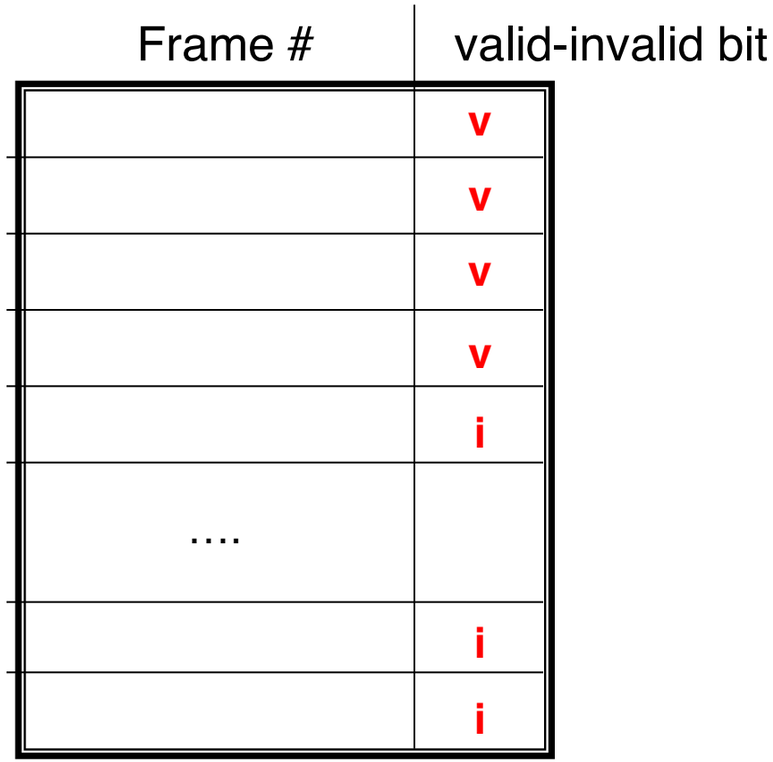
Demand Paging

- A demand paging system is quite similar to a paging system with swapping where processes reside in secondary memory and pages are loaded only on demand, not in advance.
- When a context switch occurs, the operating system does not copy any of the old program's pages out to the disk or any of the new program's pages into the main memory Instead, it just begins executing the new program after loading the first page and fetches that program's pages as they are referenced.
- While executing a program, if the program references a page which is not available in the main memory because it was swapped out a little ago, the processor treats this invalid memory reference as a **page fault** and transfers control from the program to the operating system to demand the page back into the memory.



Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v** ⇒ in-memory – **memory resident**, **i** ⇒ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:



- During MMU address translation, if valid–invalid bit in page table entry is **i** ⇒ page fault

PAGE TABLE WHEN SOME PAGES ARE NOT IN MAIN MEMORY

LOGICAL MEMORY

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

PAGE TABLE

	FRAME	VALID–INVALID BIT
0	4	v
1	–	i
2	6	v
3	–	i
4	–	i
5	9	v
6	–	i
7	–	i

v = valid (in memory)

i = invalid (not in memory)

PHYSICAL MEMORY

0	
1	
2	
3	
4	A
5	
6	C
7	
8	
9	F
10	
11	
12	
13	
14	
15	

SECONDARY STORAGE (DISK)

v (valid): The page is in main memory.

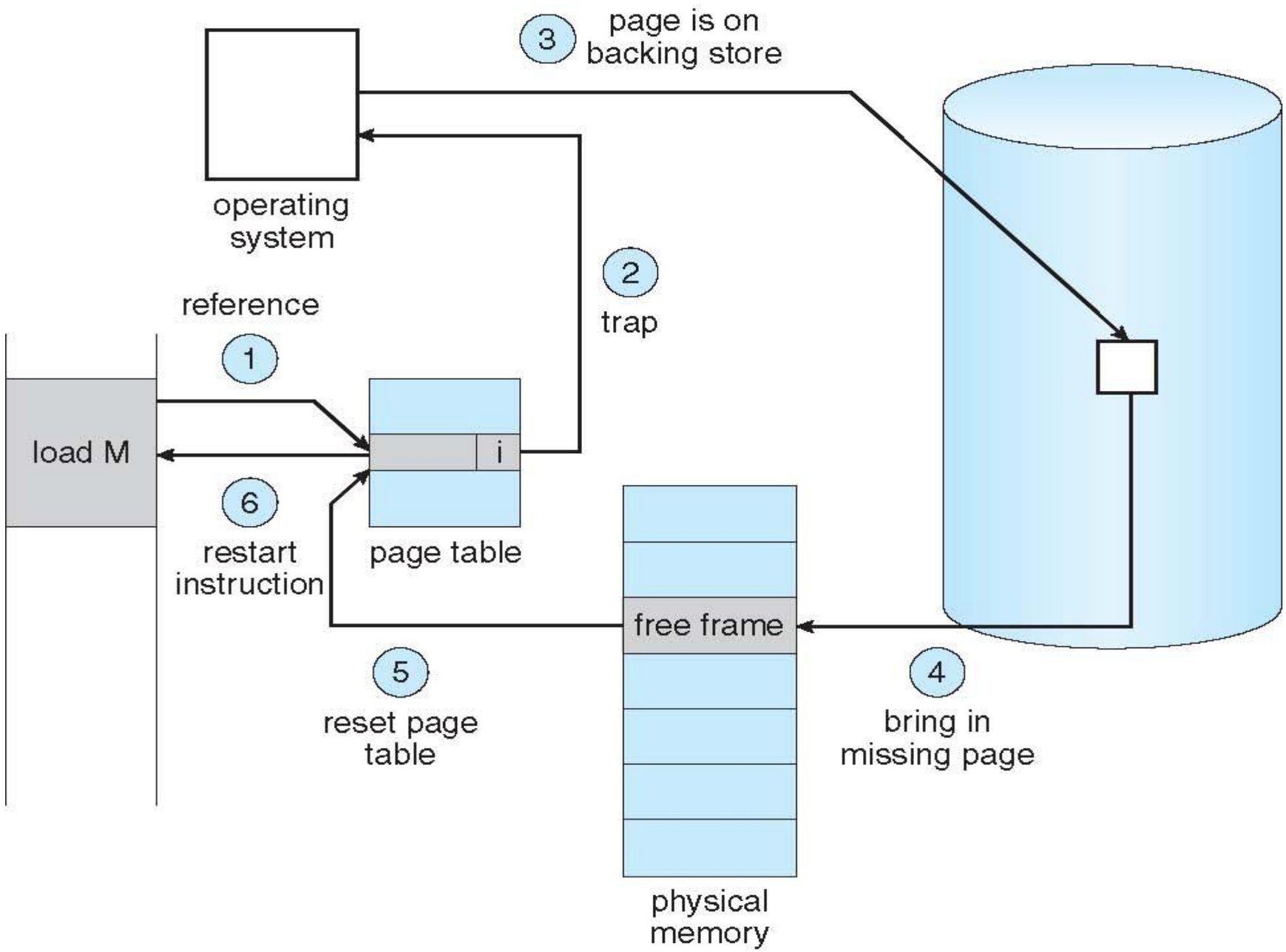
i (invalid): The page is not in main memory (on disk).

Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system: **page fault.**

1. Operating system looks at another table to decide:
   - Invalid reference ⇒ abort
   - Just not in memory
2. Find free frame
3. Swap page into frame via scheduled disk operation
4. Reset tables to indicate page now in memory
   Set validation bit = **v**
5. Restart the instruction that caused the page fault

Steps in Handling a Page Fault



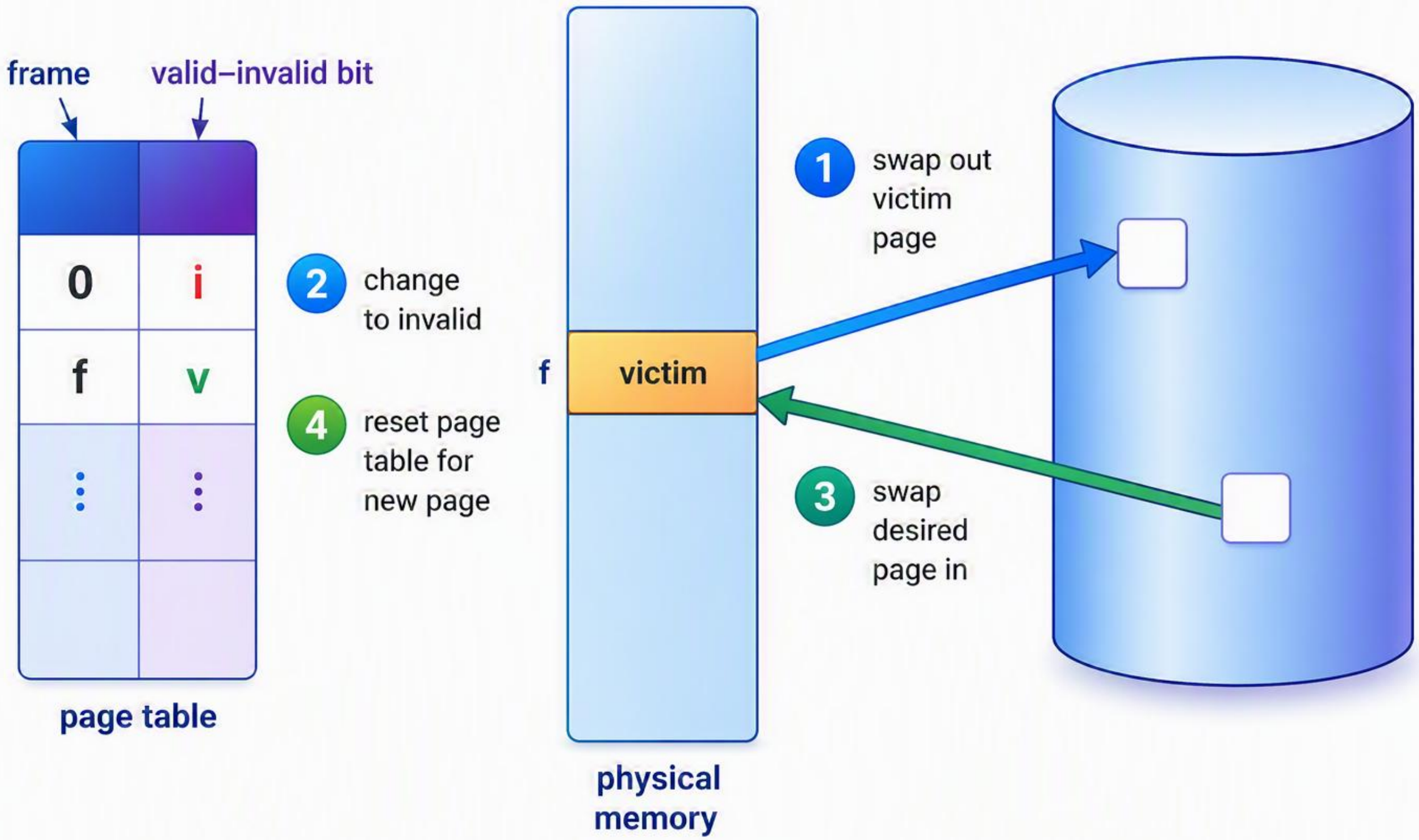
What Happens if There is no Free Frame?

- **Page replacement** – find some page in memory, but not really in use, page it out.
  - Algorithm –replace the page.
  - Performance – want an algorithm which will result in minimum number of page faults.
- Same page may be brought into memory several times.

Basic Page Replacement

1. Find the location of the desired page on disk.
2. Find a free frame:
   - If there is a free frame, use it.
   - If there is no free frame, use a page replacement algorithm to select a victim frame.
     - Write victim frame to disk if dirty.
3. Bring the desired page into the (newly) free frame; update the page and frame tables.
4. Continue the process by restarting the instruction that caused the trap.

Page Replacement



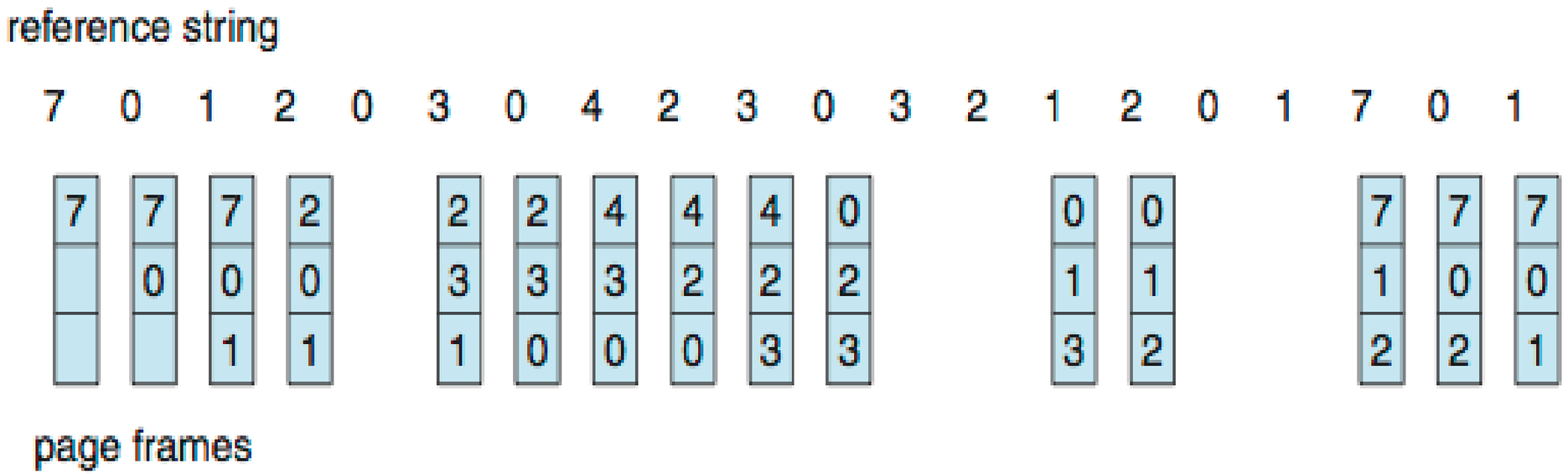
Page Replacement Algorithms

- **Page-replacement algorithm**
  - Want lowest page-fault rate on both first access and re-access.
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string.
  - String is just page numbers, not full addresses.
  - Repeated access to the same page does not cause a page fault.
  - Results depend on number of frames available.
- In all our examples, the **reference string** of referenced page numbers is:

  7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

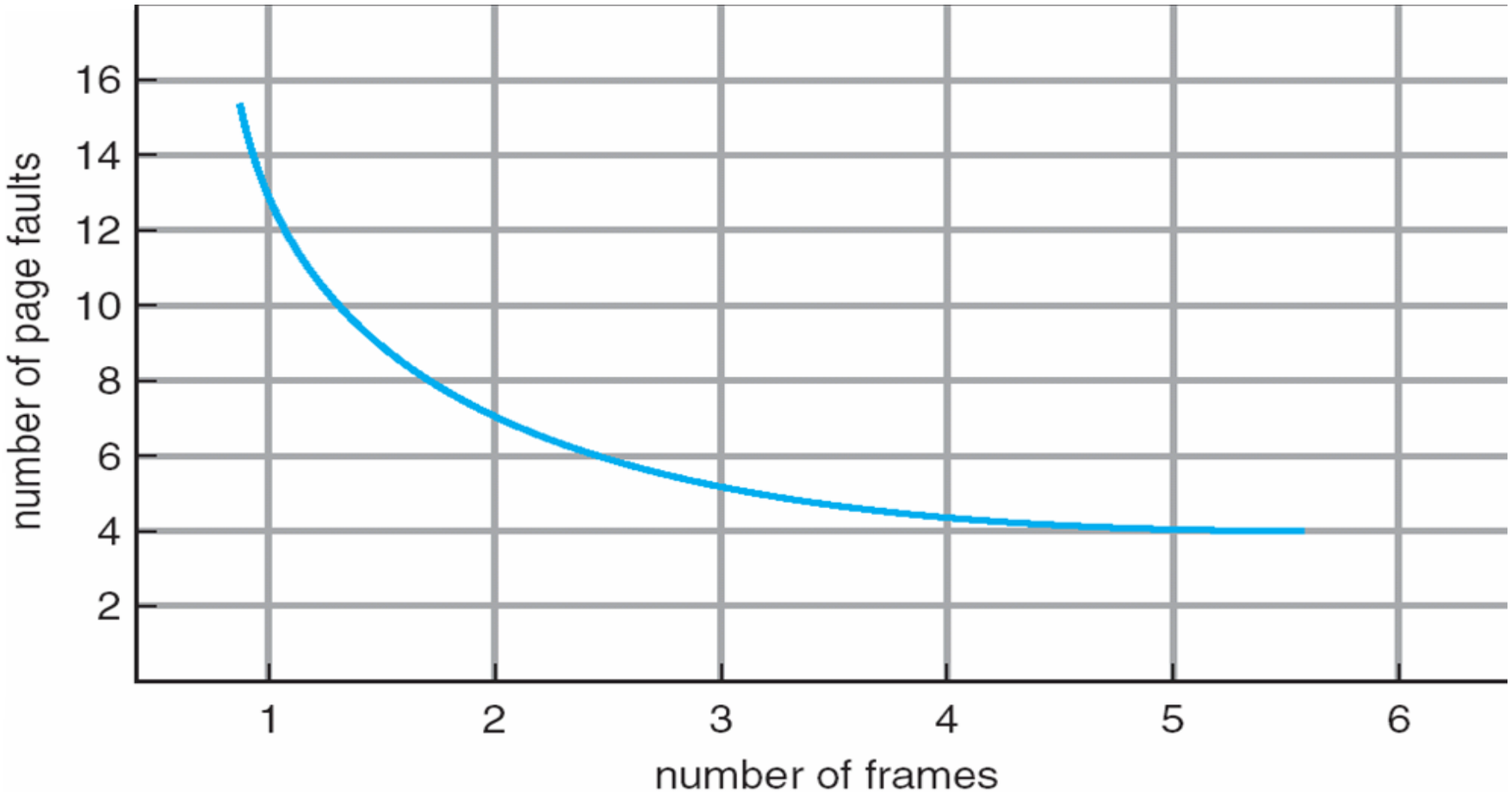
First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1.**
- 3 frames (3 pages can be in memory at a time per process).



15-page faults

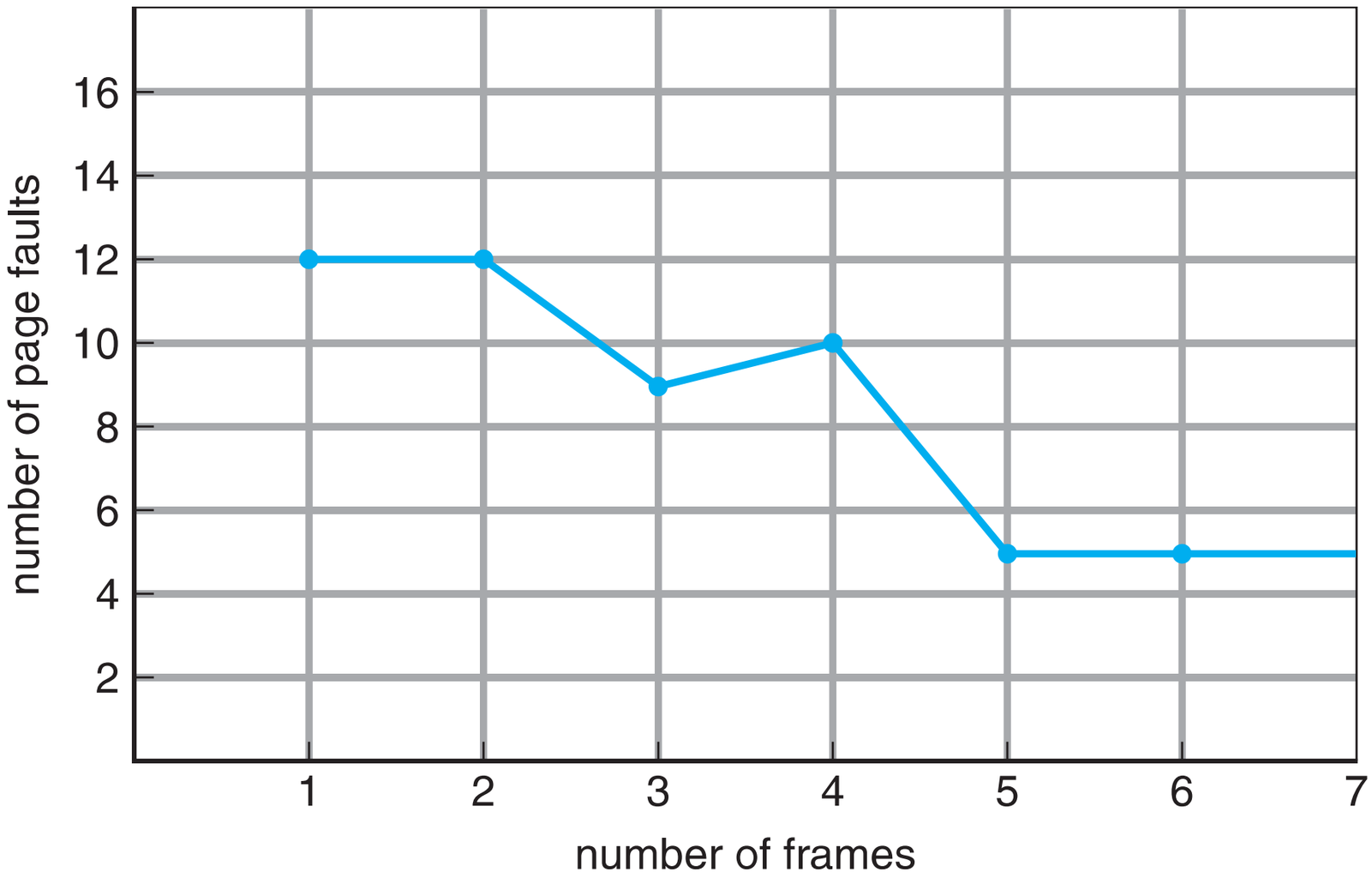
Page Faults vs. Number of Frames



FIFO Illustrating Belady's Anomaly

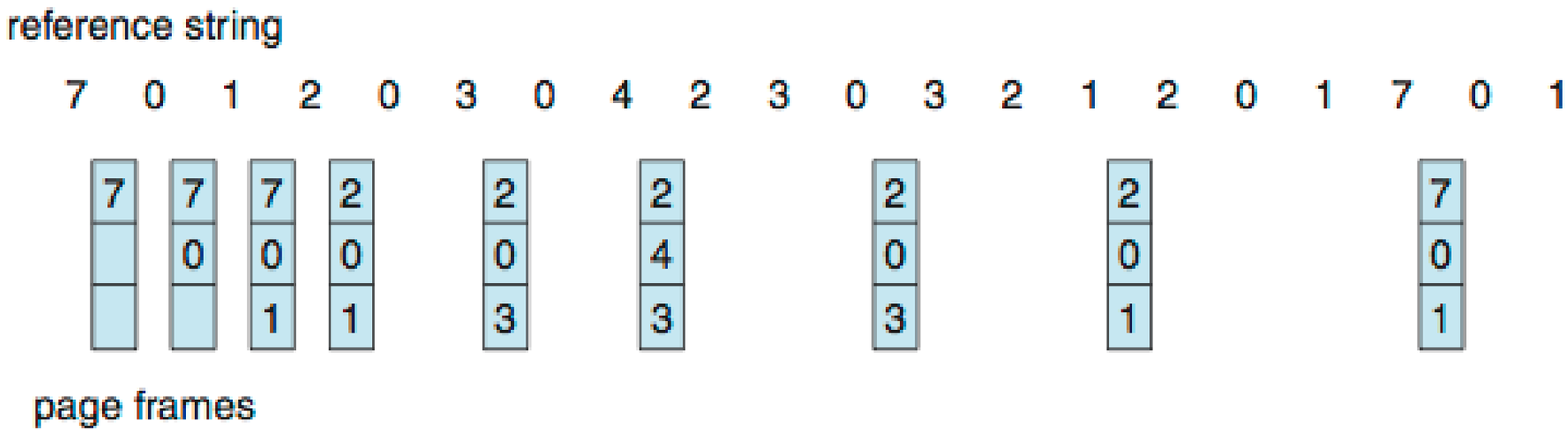
Belady's Anomaly: Adding more frames sometimes can cause more page faults!

Consider: 1,2,3,4,1,2,5,1,2,3,4,5



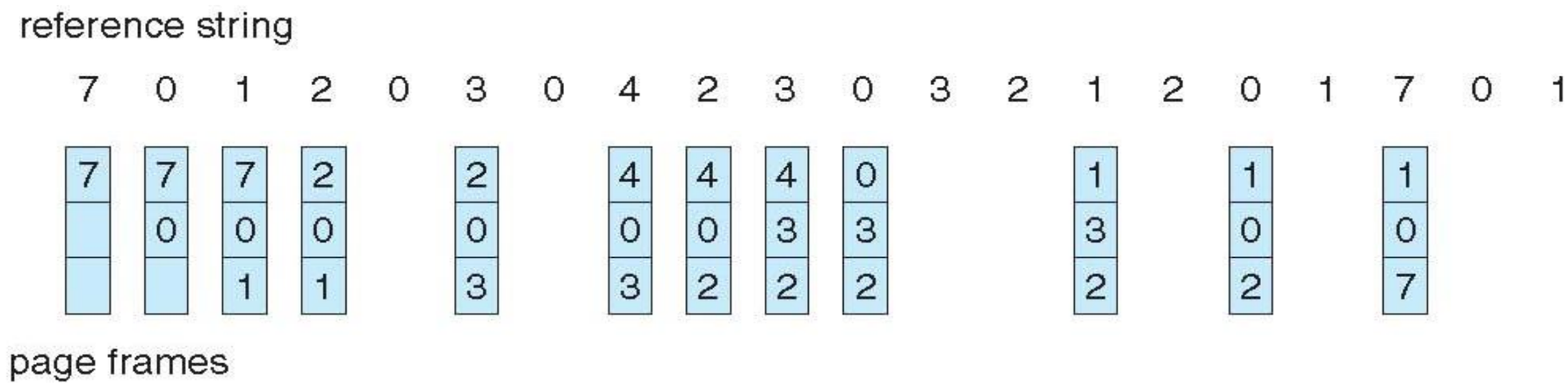
Optimal Algorithm

- Replace page that will not be used for longest period of time.
- How do you know this?
  - Not easy to read the future.



Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future.
- Replace page that has not been used in the most amount of time.



Thrashing

- At any given time, only a few pages of any process are in the main memory and therefore more processes can be maintained in memory.
- Furthermore, time is saved because unused pages are not swapped in and out of memory.
- However, the OS must be clever about how it manages this scheme. In the steady-state practically, all of the main memory will be occupied with process pages, so that the processor and OS have direct access to as many processes as possible.
- Thus, when the OS brings one page in, it must throw another out. If it throws out a page just before it is used, then it will just have to get that page again almost immediately.
- Thrashing is a condition in a virtual memory system where the operating system spends more time swapping pages between RAM and disk than actually executing processes. As a result, system performance drops drastically.
- So, a good page replacement algorithm is required.

THRASHING

The system spends more time swapping pages than executing processes

1 Program Needs Page
A process requests a page that it wants to access.

2 Page Not in RAM
The required page is not present in main memory.

3 Page Fault
Operating system is notified of the missing page.

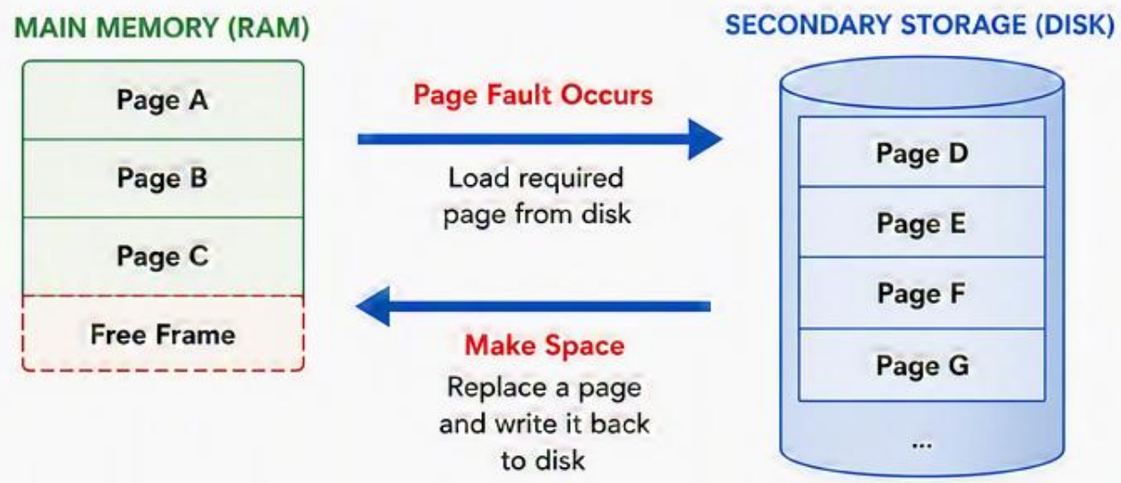
4 Load Page from Disk
OS loads the required page from secondary storage (disk) into RAM.

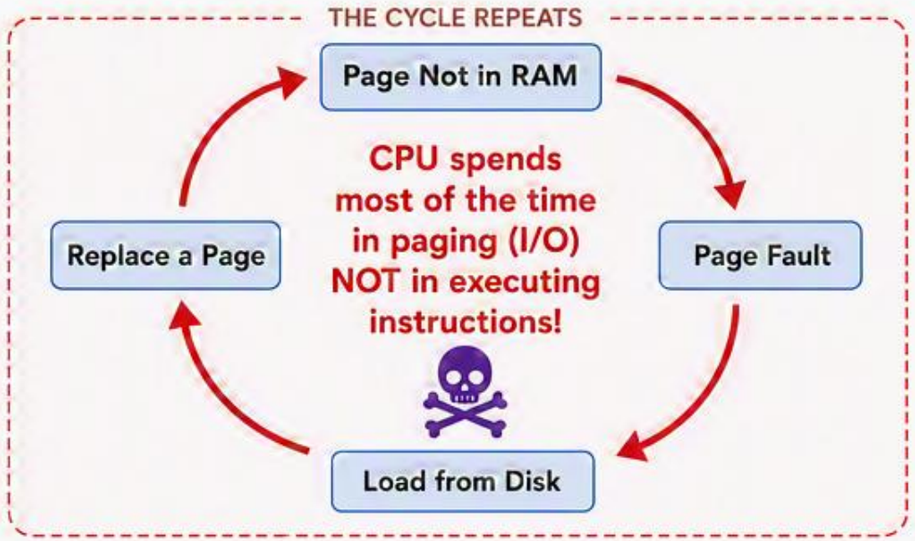
5 Replace Another Page
To make space, another page is removed from RAM and sent back to disk.

6 Next Page Needed Again
Soon, another page is needed which is also not in RAM.

7 Repeated Page Faults
The cycle repeats again and again.

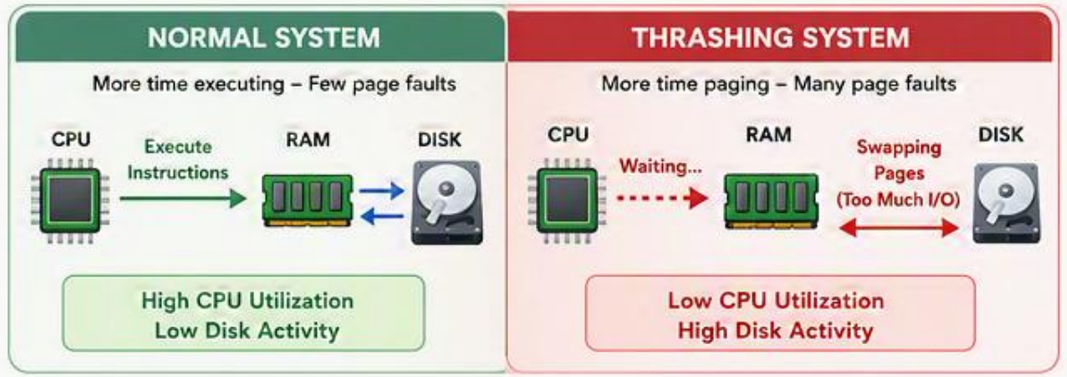
8 THRASHING
System spends most of the time swapping pages instead of doing useful work.





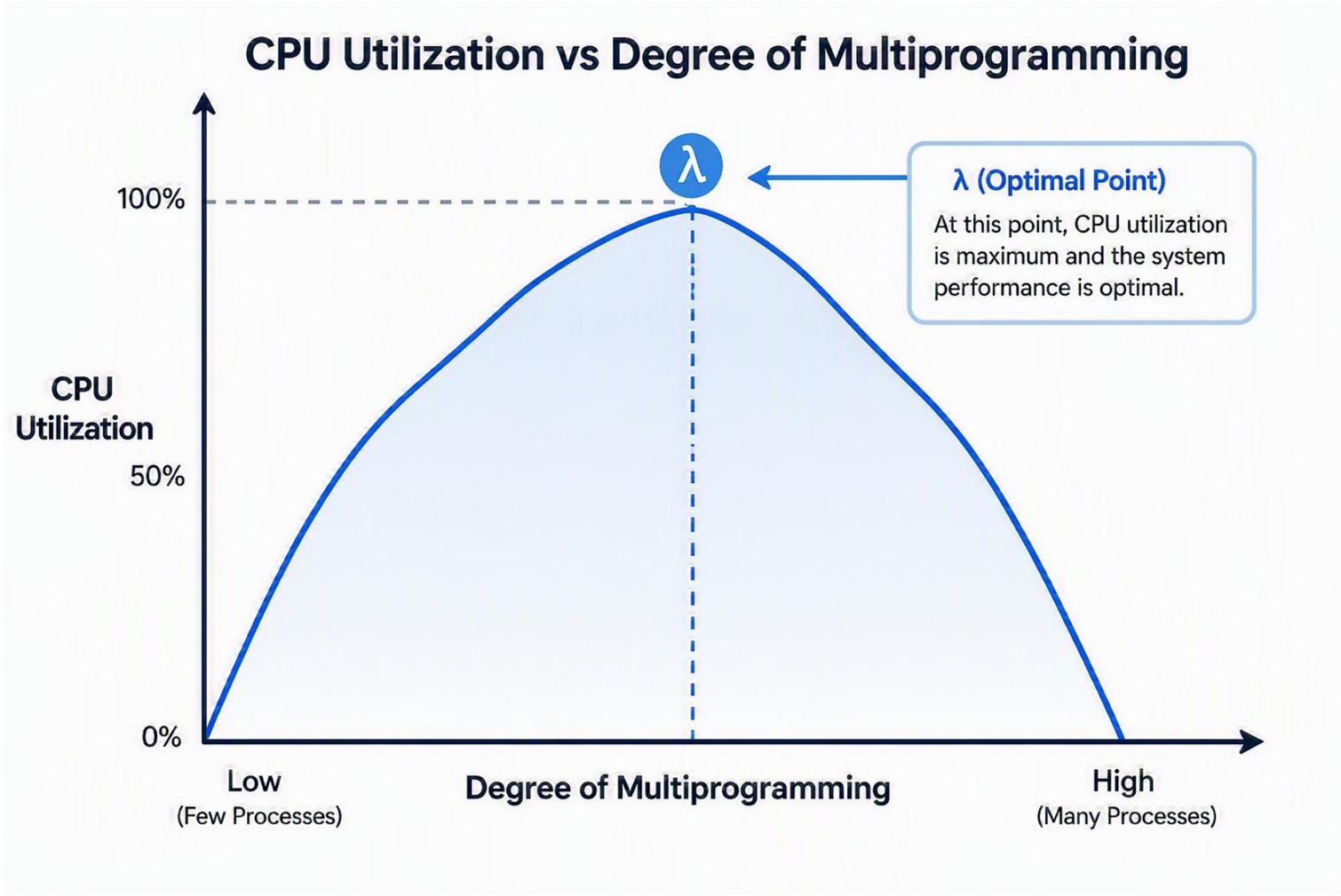
EFFECTS OF THRASHING

- Very high page fault rate
- Low CPU utilization
- High disk I/O
- Slow system performance
- Poor user experience



HOW TO PREVENT THRASHING

- Increase physical memory
- Reduce the number of running processes
- Use efficient page replacement algorithms
- Control the degree of multiprogramming



- In the given diagram, the initial degree of multiprogramming up to some extent of point(lambda), the CPU utilization is very high and the system resources are utilized 100%.
- But if we further increase the degree of multiprogramming the CPU utilization will drastically fall down and the system will spend more time only on the page replacement and the time is taken to complete the execution of the process will increase. This situation in the system is called thrashing.

Causes of Thrashing

- **High degree of multiprogramming** : If the number of processes keeps on increasing in the memory then the number of frames allocated to each process will be decreased. So, fewer frames will be available for each process. Due to this, a page fault will occur more frequently and more CPU time will be wasted in just swapping in and out of pages and the utilization will keep on decreasing. For example:

  Let free frames = 400.

  - **Case 1**: Number of process = 100.
  - Then, each process will get 4 frames.
  - **Case 2**: Number of processes = 400 .
  - Each process will get 1 frame.
  - Case 2 is a condition of thrashing, as the number of processes is increased, frames per process are decreased. Hence CPU time will be consumed in just swapping pages.

- **Lacks of Frames**: If a process has fewer frames then fewer pages of that process will be able to reside in memory and hence more frequent swapping in and out will be required. This may lead to thrashing. Hence sufficient amount of frames must be allocated to each process in order to prevent thrashing.

Recovery of Thrashing

- Do not allow the system to go into thrashing by instructing the long-term scheduler not to bring the processes into memory after the threshold.
- If the system is already thrashing, then instruct the mid-term scheduler to suspend some of the processes so that we can recover the system from thrashing.

Paging vs. Segmentation

Paging	Segmentation
Paging divides program into fixed size pages.	Segmentation divides program into variable size segments.
OS is responsible	Compiler is responsible.
Paging is faster than segmentation	Segmentation is slower than paging
Paging is closer to Operating System	Segmentation is closer to User
It suffers from internal fragmentation	It suffers from external fragmentation
Logical address is divided into page number and page offset	Logical address is divided into segment number and segment offset
Page table is used to maintain the page information.	Segment Table maintains the segment information

Segmented Paging

- Pure segmentation is not very popular and not being used in many of the operating systems.
- However, Segmentation can be combined with Paging to get the best features out of both the techniques.
- In Segmented Paging, the main memory is divided into *variable size segments* which are further divided into ***fixed size pages***.
- Pages are smaller than segments.
- Each Segment has a page table which means every program has multiple page tables.
- The logical address is represented as Segment Number (base address), Page number and page offset.
  - **Segment Number** → It points to the appropriate Segment Number.
  - **Page Number** → It Points to the exact page within the segment.
  - **Page Offset** → Used as an offset within the page frame.

Continue…

Advantages of Segmented Paging

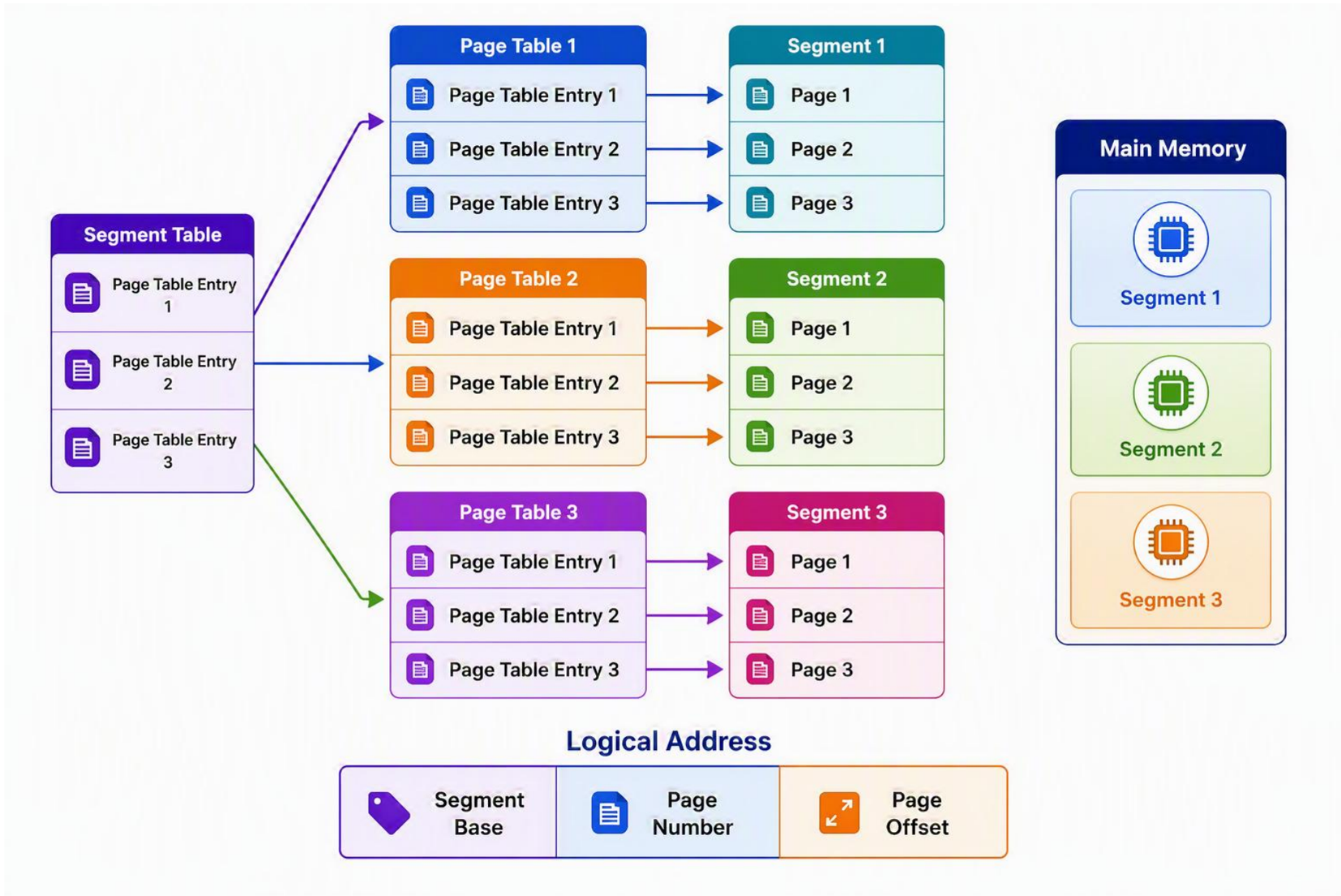
- It reduces memory usage.
- Page table size is limited by the segment size.
- Segment table has only one entry corresponding to one actual segment.
- External Fragmentation is not there.
- It simplifies memory allocation.

Disadvantages of Segmented Paging

- Internal Fragmentation will be there.
- The complexity level will be much higher compared to paging.
- Page Tables need to be contiguously stored in the memory.

Continue…

- Each Page table contains the various information about every page of the segment.
- The Segment Table contains the information about every segment. Each segment table entry points to a page table entry and every page table entry is mapped to one of the page within a segment.



Continue…

Translation of logical address to physical address

- The CPU generates a logical address, which is divided into two parts: Segment Number and Segment Offset.
- The Segment Offset must be less than the segment limit. Offset is further divided into Page number and Page Offset.
- To map the exact page number in the page table, the page number is added into the page table base.
- The actual frame number with the page offset is mapped to the main memory to get the desired word in the page of the certain segment of the process.

Segment Table

Segment Limit	Page Table Base
⋮	⋮
⋮	⋮

Legend

- S → Segment Table
- SO → Segment Offset
- P → Page Number
- PO → Page Offset
- f → Frame Number

CPU → S | SO → S → < → Yes → SO → P | PO

< → No → Invalid Address

P | PO → + → Frame → f | PO → Physical Memory

Steps:

1. CPU generates logical address (S, SO)
2. Check if SO < Segment Limit
3. Use Page Table Base to access segment's page table
4. Add Page Base (f) with Page Offset (PO) to get physical address

Page Table
(For Each Segment)

Physical Memory

Segment Table

Segment	Limit	Page Table Base
S0	10	1000
S1	50	1200
S2	20	1400

0

CPU → S | d

0, 4

Logical Address

4 → d<limit

yes → d

no → Error

10

4

2 | 0

p | po

F | po

1, 0

Page Size=2

0	P0
1	
2	P1
3	
4	P2
5	
6	P3
7	

S0

P0	1000	F3
P1	1001	F2
P2	1002	F1
P3	1003	F0

Page Table

0	P3	F0
1	P2	F1
2	P1	F2
3	P0	F3
4		.
5		.
6		.
7		Fn

Page Number=d(offset)/page size
=4/2=2

Page offset(po)= d % page size
=4%2=0

Physical address=(frame no*page size)+o
=1*2+0=2

Practice

- Considering the following segment table and page table, translate the following logical address to physical address: (2, 6), where page size=2.

Segment Table

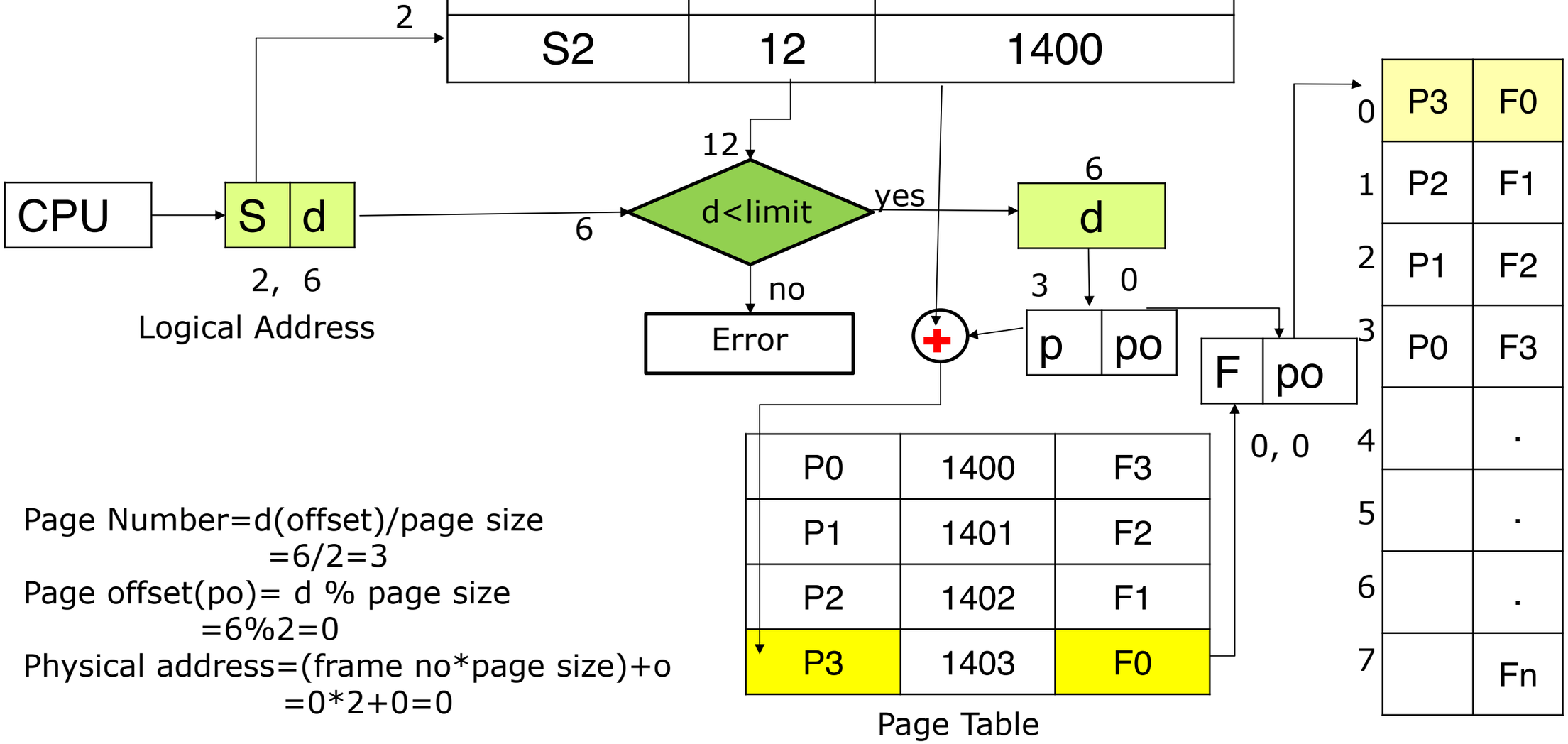
Segment	Limit	Page Table Base
S0	10	1000
S1	14	1200
S2	12	1400

Page Table

Page	Address	Frame
P0	1400	F3
P1	1401	F2
P2	1402	F1
P3	1403	F0

Segment Table

Segment	Limit	Page Table Base
S0	10	1000
S1	14	1200
S2	12	1400



Page Size=2

0	P0
1	
2	P1
3	
4	P2
5	
6	P3
7	

S0

Page Number=d(offset)/page size
=6/2=3
Page offset(po)= d % page size
=6%2=0
Physical address=(frame no*page size)+o
=0*2+0=0

SEGMENTED PAGING ADDRESS TRANSLATION

INPUT FROM CPU

LOGICAL ADDRESS

Segment (S)	Offset (d)
2	6

(2, 6)

Page Size = 2

1 SEGMENT TABLE

Segment	Limit	Page Table Base
S0	10	1000
S1	14	1200
S2	12	1400

Segment = 2 (S2)
Limit = 12
Offset (d) = 6
Check: d < limit
6 < 12 ✓ Valid

2 CHECK LIMIT

Is d < limit?
6 < 12 ✓ Yes

3 FIND PAGE NUMBER AND OFFSET

Page Number (p) = [d / Page Size]
= [6 / 2] = 3

Page Offset (po) = d % Page Size
= 6 % 2 = 0

Result

p = 3
po = 0

4 LOCATE PAGE TABLE ENTRY

Page Table Base (for S2) = 1400
Page Table Entry Address
= Base + p
= 1400 + 3 = 1403

5 PAGE TABLE (for S2)

Page	Address (Base + p)	Frame
P0	1400	F3
P1	1401	F2
P2	1402	F1
P3	1403	F0

At address 1403 we get Frame F0

PHYSICAL MEMORY (FRAMES)

0	P0
1	P0
2	P1
3	P1
4	P2
5	P2
6	P3
7	P3
	...
	Fn

Frame F0

6 PHYSICAL ADDRESS

Physical Address = (Frame, po)
= (F0, 0)

Linear Address (optional)
= Frame × Page Size + po
= 0 × 2 + 0 = 0

SUMMARY

1. From Segment Table, get limit and base.
2. Check if offset < limit → Valid.
3. Calculate page number (p) and page offset (po).
4. Find page table entry using base + p.
5. Get frame number from page table.
6. Physical Address = (Frame, po) = (F0, 0)

FINAL ANSWER

Physical Address = (F0, 0)

Legend

d = Offset, p = Page Number, po = Page Offset
F0 = Frame 0, Page Size = 2

THANK YOU!